More Parameters Undo what Previous Params Did: An analysis on Representational Kinematic Redundancy Using Infinite Interpolation Over Residual Blocks Without Retraining

Jaume Ivars Grimalt [1]

Abstract

The abstract must contain a brief intro of the field you are attacking on, the phenomenon you observed, an explanation and an interpretation. Code is available at https://github.com/jaume2000/dynamics_skip_nn.

1. Introduction

A residual block $x + f(x)$ has the algebraic form of one Euler step of size 1. That observation has been used to read residual networks as discretisations of an ODE, but a trained stack need not be a *smooth* field in depth: Figure 1 shows two ConvNeXt stage-3 transports of the same horizon $T{=}9$. Sharing one block yields a single autonomous flow whose speed rises and falls as a bell; independent blocks produce a new field every unit of time, with kinks and speed jumps at every integer. The rest of the paper makes that contrast quantitative—and asks how much of the accuracy of the switched stack is already carried by the shared field.

Skip Connections-NN (SCNN) wanted to solve grad vanishing problem that VGG introduced. By defining $x = f(x) + x$ it induced some properties: Works in the same feature representation. P.e. A given channel will have the same purpose in the next layer.

(Sander et al., 2022; Queiruga et al., 2020) says that resnets just have the algebraic form of Euler ODE solver, that if params are steps they won'r be ODEs. We apport evidence that residual blocks are 1-step ODE solvers and we therefore have a sum of Ode solvers that are constantly changing of domain and we can accomplish the same dynamics by sharing the weights and we augmentate the resolution of the feature dynamics to study with better quality how X moves along the latent-space manifold in time / depth.

Questions we want to solve:

- Can SCNN's blocks be looped indefinitely? → Yes
- Are all this weights necessary? + Shared networks can reach similar performance? → No + Yes
- What are the dynamics of the features in this two types of models? Statement: The over-parametrized networks undo what the previous layer does, that's why SCNNs and weight-tied are equal. Tangential acceleration is crucial for this affirmation. Here we analyze the geometry of trajectories. Take into account the massive activations here.
- How can we improve discrete models given this evidence? -¿ Delta networks?

2. Related Work

3. Method

We study residual vision networks as a three-part map. The first part—stem, stage 1 and stage 2—encodes the image into a latent state x_0; the second part—the deepest residual stage—transports x_0 to a state x_T ready for decoding; the third part—stage 4 and the prediction head—maps x_T to class logits. We focus on that middle transport (stage 3 in ConvNeXt, Swin and ResNet). It holds most of the residual depth, keeps a fixed representation and spatial resolution, and is the stage with enough blocks to study dynamics: in ConvNeXt-T, for example, stages 1, 2 and 4 have three blocks each, while stage 3 has nine. ConvNeXt inherits this depth layout from Swin, itself patterned on ResNet, so the same middle stage is the natural object of study in all three.

In ConvNeXt-T we take that stage's $n{=}9$ blocks as the trained horizon $T{=}n{=}9$, and refine the discretisation with repetitions R and step $\Delta t{=}1/R$ (Section 3.2). Effective per-block step sizes may differ, but they are absorbed into the learned weights, so integrating each block over one unit of time is equivalent to the steps the model has already internalised.

The central question is functional: can a weight-tied stage 3—one shared residual field reused over the horizon $T{=}n$—match the representational power of the non-tied

[1]Mycrospace AI Research, Valencia, Spain. **AUTHORERR: Missing \icmlcorrespondingauthor.**

arXiv preprint.

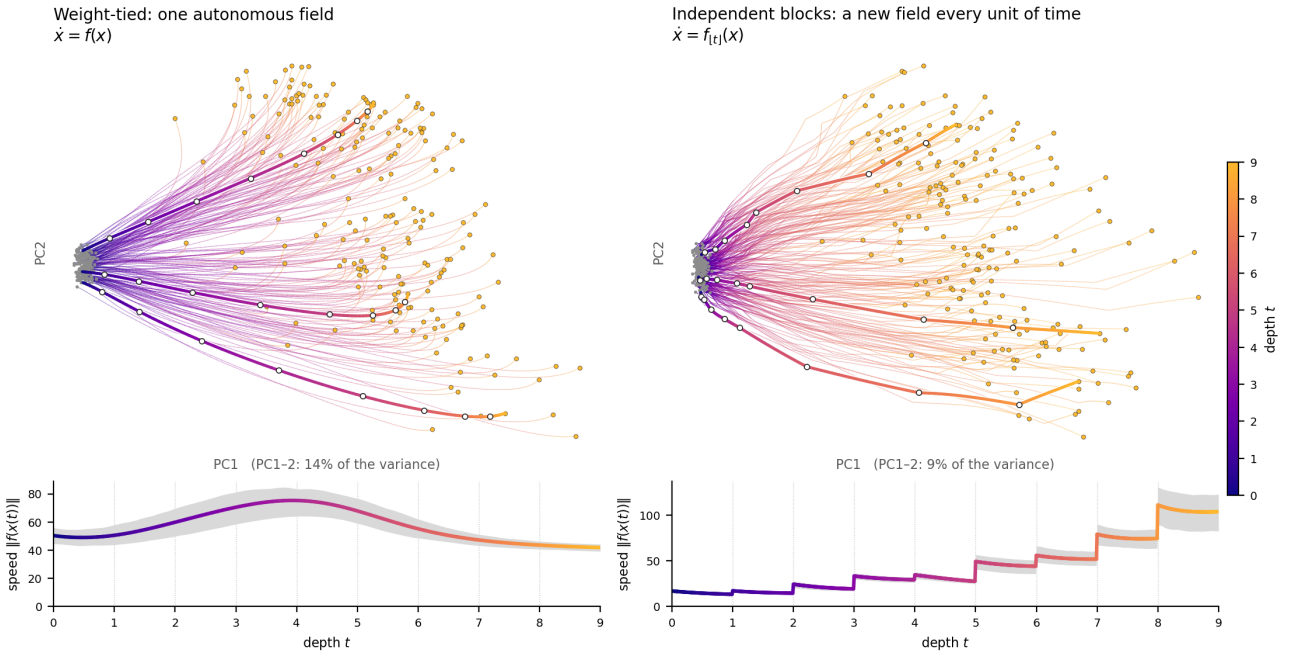


Figure 1. Stage-3 feature trajectories of a weight-tied ConvNeXt (left) and a standard ConvNeXt-T (right), refined at inference with Δt=0.01 (T=9; massive channel removed). **Top:** PCA of all states, paths coloured by depth t; white dots mark integer t. **Bottom:** residual speed $\|f(x(t))\|$ (class means; shade: spread). One shared block is a single autonomous field $\dot{x} = f(x)$; nine independent blocks switch the field at every integer, $\dot{x} = f_{\lfloor t \rfloor}(x)$. We analyse this contrast throughout the paper.

stage, which uses n independent blocks and many more parameters? Simply looping the blocks at finer step size does not answer this. Residual maps are Lipschitz, so refining the discretisation at fixed T always converges to a continuous limit, trained or randomly initialised (Appendix D); convergence alone is therefore a weak claim. What the shared construction does give us is a clean autonomous ODE for stage 3. We then read the standard network the same way, block by block: each residual update is a one-step Euler step of a local field, so the untied stage is a switched composition of smooth ODE pieces rather than a single autonomous flow—hence the jumps in $\|f\|$ at block boundaries in Figure 1. Continuising both—tied and untied—at inference yields high-resolution trajectories from x_0 to x_T, and those trajectories are where the two models diverge: the standard path repeatedly redirects and partially undoes itself, while the tied path stays closer to a monotone transport. We organise the section accordingly:

1. Section 3.1 shows that a weight-tied stage 3 nearly matches an untied ConvNeXt-T (80.36% vs. 80.97% top-1), so one shared field already carries most of the function;

2. Section 3.2 continuises residual stages at inference without fine-tuning—ConvNeXt-T and its weight-tied variant, and also Swin-T and ResNet-50/101—showing the same pattern across architectures: at fixed T every solver reaches one limit and retains almost all accuracy, whereas breaking T collapses the networks;

3. Section 3.3 compares the refined trajectories, autonomous (tied) versus switched (untied);

4. Section 3.4 measures that pull-and-push along depth, and probes when stage 3 prefixes become usable by a frozen stage 4+head;

5. Section 3.5 tests a hypothesis: small per-block corrections on a shared field as a way to recover what over-parameterisation wastes.

3.1. Do weight-tied stage-3 networks achieve similar results?

We first ask whether the stage-3 transport needs nine independent blocks, or whether one shared block applied nine times is enough. Sun et al. (2025) study a related question in transformers: they replace and loop layers with a middle block to test whether all layers implement the same operation. We observe the same phenomenon in ConvNeXt stage 3—ablations that permute, duplicate or substitute blocks are reported in Appendix E—and go one step further:

if the stage can be run with a single repeated function, a network trained to use that shared function should reach a similar accuracy level.

We train a vanilla ConvNeXt-T and a weight-tied stage-3 variant under the same recipe (ImageNet-1k, batch size 1024; configuration in Appendix A). Our baseline reaches 80.97% top-1 (28.59M params), slightly below the original ConvNeXt-T report of 82.1% (Liu et al., 2022)—expected under our hardware constraints—but all comparisons below use models trained by us under matched conditions.

The weight-tied model (18.97M params, 33% fewer) reaches 80.36% top-1 (-0.61 points). Sharing all four stages (8.68M params, 70% fewer) reaches 79.01% (-1.96 points).

A network that implements (approximately) the same residual function at every depth can therefore sit close to the untied baseline. That motivates the next subsection—not as a recipe for a deeper or larger network, but as an inference-time probe: if stage 3 is (approximately) an Euler discretisation, refining it with step $\Delta t{=}1/R$ at fixed $T{=}9$ should keep a decoder-usable state and, equally important, yield high-resolution trajectories for analysis. We apply that probe to the weight-tied model and to standard residual architectures.

3.2. Indefinite looping without fine-tune

Section 3.1 showed that one shared block nearly matches a standard ConvNeXt stage 3 (independent per-block weights). We now refine residual blocks at inference as one-step Euler updates of a vector field: repeat each update R times with step $\Delta t{=}1/R$, keep the trained horizon $T{=}n$, and do not fine-tune. The goal is not to replace a small network by a R-times deeper one that trades a fraction of a point of accuracy for width in depth—that would be the wrong reading. Refinement is a probe with two uses. First, it tests whether the continuous limit stays functionally close to the native model for stage 4 and the head: on ConvNeXt-T the refined endpoint sits $\sim 21^\circ$ from the $R{=}1$ state (Figure 5), yet top-1 barely moves, so the decoder still sees a usable representation. Second, it raises the temporal resolution of the path from x_0 to x_T, which is what later geometry and undoing analyses need. Convergence of the refinement by itself is not diagnostic—any Lipschitz residual has a continuous limit (Appendix D)—so what matters is what the limit retains and how the trajectory is organised. We apply the probe to weight-tied and standard networks (ConvNeXt, Swin, ResNet); trajectory geometry is deferred to Section 3.3.

A residual block computes $x + f(x)$ (He et al., 2016). With step size 1, that is exactly one step of the explicit Euler method for the ODE $\dot{x} = f(x, t)$ (E, 2017; Haber & Ruthotto, 2017; Lu et al., 2018). Write the residual of block i as

$$f_i(x) \;=\; \mathrm{block}_i(x) - x. \tag{1}$$

Then the native network is one Euler step of size 1 per block. Over a stage of n blocks (e.g. $n{=}9$ in ConvNeXt-T stage 3), starting from the output x_0 of the previous stage,

$$x_{i+1} \;=\; x_i + f_i(x_i), \qquad x_n \;=\; x_0 + \sum_{i=0}^{n-1} f_i(x_i). \tag{2}$$

Some argue that this algebraic form is already enough to call the network an Euler solver (E, 2017; Haber & Ruthotto, 2017; Lu et al., 2018), but algebraic form alone is not sufficient evidence: Queiruga et al. (2020) argue that ResNets fail to be meaningful dynamical integrators beyond that syntactic similarity, and Sander et al. (2022) show that a ResNet approaches its associated Neural ODE only when the residual maps vary smoothly in depth (otherwise the discrete stack need not be a consistent discretization of a single continuous flow).

We suggest reading the stage as a *sum of several ODEs*, one per block, where the network is trained to solve each unit interval with a single Euler step. The continuous object behind (2) is therefore

$$x_n \;=\; x_0 + \sum_{i=0}^{n-1} \int_i^{i+1} f_i\big(x(t)\big)\, dt, \tag{3}$$

or, block by block,

$$x_{i+1} \;=\; x_i + \int_i^{i+1} f_i\big(x(t)\big)\, dt. \tag{4}$$

Here the sum is the stack of residual blocks, and each integral is the continuous counterpart of the one-step update in (2): instead of taking one Euler step of size 1, we integrate f_i over the unit interval $[i, i+1]$, keeping the horizon of each block equal to 1. For a stage with n blocks trained at unit step, the stage horizon is therefore

$$T \;=\; n \qquad \text{(ConvNeXt-T stage 3: } n{=}9\text{, so } T{=}9\text{)}. \tag{5}$$

Repetitions R. Throughout the paper we use a single refinement parameter R (*repetitions*). For a non-tied stage, $R{=}2$ means each of the n blocks is applied twice, for nR residual updates in total. For a weight-tied stage, $R{=}2$ means the *same* shared block is applied $nR{=}18$ times. In both cases we set the Euler step from the trained horizon,

$$\Delta t \;=\; \frac{1}{R} \qquad \Rightarrow \qquad T \;=\; nR \cdot \Delta t \;=\; n, \tag{6}$$

so refining never changes the integration time—only the resolution.

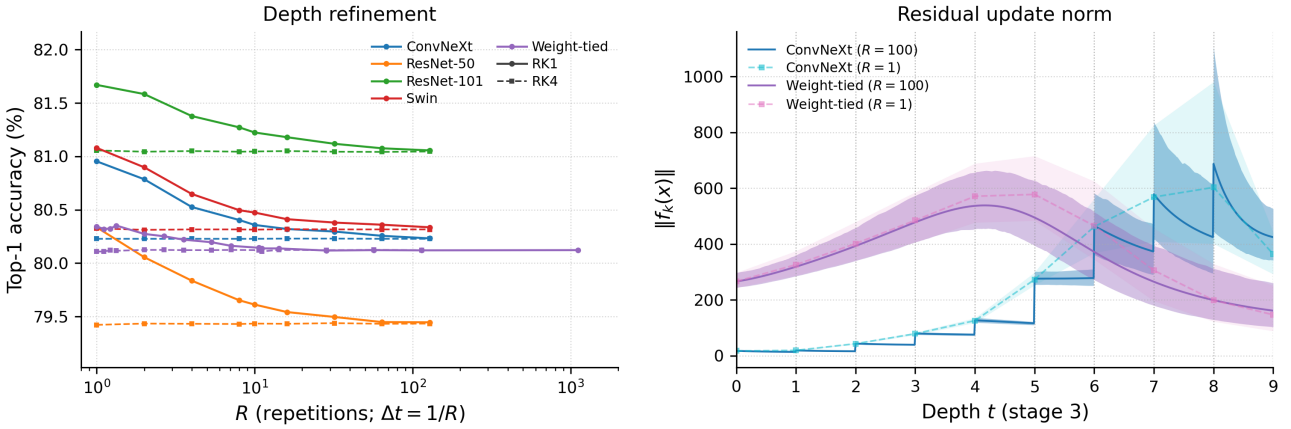


Figure 2. **Left:** top-1 vs. repetitions R (solid: Euler / RK1; dashed: RK4), with $\Delta t{=}1/R$ so $T{=}n$ is conserved (weight-tied: nR applications of the shared block). **Right:** residual-field norm $\|f(x)\|$ along stage 3 (mean and P5–P95 over 500 images). ConvNeXt is piecewise constant in depth; the weight-tied model is a single smooth autonomous field. See Section 3.2.2.

Plain interpolation keeps the field piecewise constant on each block and discretizes the integral with R Euler substeps of size $\Delta t = 1/R$:

$$\int_i^{i+1} f_i\big(x(t)\big)\,dt \;\approx\; \sum_{r=0}^{R-1} \Delta t\, f_i\big(x_{i+r\Delta t}\big). \qquad (7)$$

Equivalently, over the whole stage the refined trajectory is

$$x_T \;=\; x_0 + \sum_{i=0}^{T-1}\sum_{r=0}^{R-1} \Delta t\, f_i\big(x_{i+r\Delta t}\big), \qquad \Delta t = \tfrac{1}{R}. \quad (8)$$

When $R{=}1$, (8) recovers the original network (2). As R grows with $T{=}n$ fixed, the scheme approaches the continuous limit (3); we also compare against Midpoint (RK2) and RK4 on the same horizon.

In a weight-tied stage the residual field is the same at every depth, $f_i \equiv f$ for all i. Then (3) collapses to a single autonomous integral over $[0, T]$ with $T{=}n$:

$$x_T \;=\; x_0 + \int_0^T f\big(x(t)\big)\,dt. \qquad (9)$$

Inference uses nR applications of f with $\Delta t{=}1/R$; training uses $R{=}1$ (n steps of size 1), hence $T{=}9$ for our shared ConvNeXt-T.

3.2.1. Ablations

We apply the refinement (8) (plain interpolation, T conserved via $\Delta t{=}1/R$) to our ConvNeXt-T, our weight-tied ConvNeXt-T, Swin-T, ResNet-50 and ResNet-101 (ImageNet-1k validation, top-1). Figure 2 (left) summarises the sweep over R; numbers are in Table 1.

Accuracy decreases monotonically as R grows and plateaus at a continuous limit that matches RK4. Euler (RK1), Midpoint (RK2) and RK4 all converge to the *same* accuracy once the step is fine enough; full Mid/RK4 tables are in Appendix B. (Matched top-1 alone does not prove a shared state x_T; we check endpoint agreement of the solvers later in Appendix C.) The trained network itself (Euler, $R{=}1$) is *not* at that limit; it is the best point of every sweep. Keeping the nine steps of size $\Delta t{=}1$ but swapping Euler for RK4 already lands on the limit, both in accuracy and in the stage-3 state (Appendix C), and costs 0.6–0.9 points on the untied models (0.2 on the weight-tied one): the decoder is tuned to the state produced by a single Euler step of size 1, not to the flow it approximates.

Wrong step size. Repeating blocks without shrinking the step (e.g. $R{=}10$ with $\Delta t{=}1$, so T grows tenfold) collapses every pretrained model to chance ($\approx 0.1\%$–1% top-1). The same holds for the weight-tied network when T leaves the trained horizon (Figures 3 and 4). The Euler step must be $\Delta t{=}1/R$, equivalently $T{=}n$ for an n-block stage ($T{=}9$ when $n{=}9$). The trajectory dynamics behind this collapse are analysed in Section 3.3.5.

Weight-tied fractional $R < 1$. Figure 3 reports the shared network at fixed $T{=}9$ with fewer than nine steps ($R < 1$, i.e. $9R$ applications with $\Delta t{=}1/R > 1$); the numbers, including Midpoint, are in Table 9. Large steps need a higher-order solver: at $R{=}4/9$ (4 steps), RK4 reaches 78.9% while Euler falls to 67.8%; Euler with $R{=}2/9$ ($\Delta t{=}4.5$) is unstable (5.9%). For $R \geq 7/9$, all three solvers sit within 0.3 points of the trained baseline.

Table 1. Plain Euler refinement with $\Delta t{=}1/R$ (T conserved). Top-1 (%). For weight-tied, R means $9R$ applications of the shared block (exact $9R$ when available; else nearest measured depth*). ODE limit: RK4 at the finest step in each sweep.

R (Δt)	CNX-T	Swin-T	R-50	R-101	Tied
1 (1)	80.95	81.08	80.34	81.67	80.34
2 (0.5)	80.79	80.90	80.06	81.58	80.27
4 (0.25)	80.53	80.65	79.84	81.38	80.22*
10 (0.1)	80.36	80.47	79.61	81.22	80.14*
32 (0.03)	80.30	80.38	79.50	81.12	80.14*
128 (0.008)	80.23	80.34	79.45	81.06	80.12*
ODE limit	80.23	80.32	79.43	81.05	80.12
Gap ($R{=}1{\to}$ODE)	-0.72	-0.76	-0.91	-0.62	-0.22

*Nearest measured shared depth at $T{=}9$: 32, 96, 128, 1024 steps respectively (exact $9R$ would be 36, 90, 288, 1152).

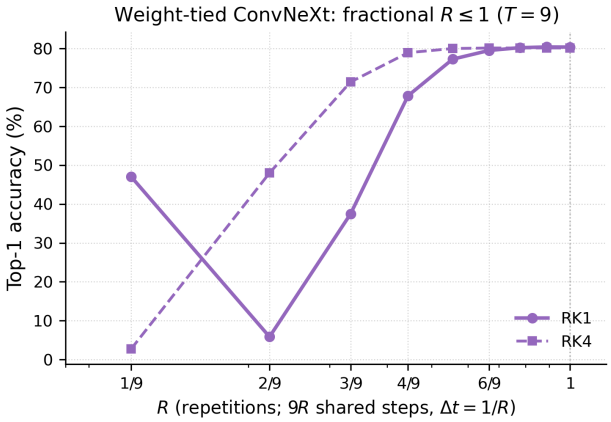


Figure 3. Shared ConvNeXt-T at fixed $T{=}9$ for fractional $R \leq 1$ (RK1 solid, RK4 dashed).

Horizon, not resolution. Varying the number of unit steps at $\Delta t{=}1$ (so T grows) or varying Δt at fixed $R{=}1$ (9 steps; $T{=}9 \cdot \Delta t$) both peak at $T{=}9$ (Figure 4): the classification head expects the state reached after that integration time. Integrating too short or too long degrades smoothly near $T{=}9$ and collapses far from it. We conclude that the best Euler step is the one that preserves the trained horizon: $\Delta t{=}1/R$ for every model (non-tied and weight-tied).

Do refined blocks land on the same x? Near-baseline accuracy does *not* mean that R Euler steps of size $1/R$ reproduce the unit-step state. Figure 5 compares the refined ConvNeXt-T trajectory ($R{=}100$, $\Delta t{=}0.01$) with the native $R{=}1$ trajectory on the same images, in ℓ_2 distance (top, next to both state norms) and cosine similarity (bottom) (massive channel removed from the metrics, see Section 3.3.1; $N{=}500$ images). Three facts stand out.

- **Within each block** the refined trajectory moves *toward* the parallel $R{=}1$ checkpoint: the distance decays and the cosine rises until the block ends, so the refinement is not drifting away from the one-step endpoint of that block.

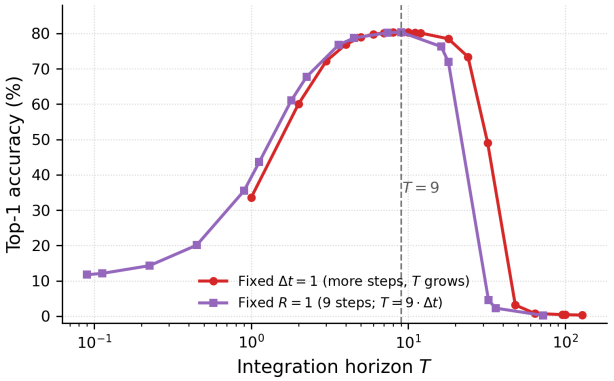


Figure 4. Shared ConvNeXt-T: top-1 vs. horizon T. Red: fixed $\Delta t{=}1$, more steps (T grows). Purple: fixed $R{=}1$ (9 steps), varying Δt ($T{=}9 \cdot \Delta t$). Both peak at the trained horizon $T{=}9$.

- **Across blocks** the gap grows: the block-end distance rises from ≈ 5.6 to ≈ 76 at $t{=}9$, against $\|x\| \approx 203$ at the stage exit, i.e. a relative gap $\|x^{(R)} - x_{R=1}\|/\|x\| \approx 0.36$.

- **The gap is a rotation.** The block-end cosine drops from 0.99 to 0.93 ($\sim 21°$), while $\|x\|$ differs by only $\approx 3\%$ (203 vs. 210); a pure rescaling of that size would account for a distance of ≈ 7, not 76.

Each block therefore implements a *different* map than its one-step counterpart once the input has already drifted; local refinement stays contractive toward the current block's $R{=}1$ target, but errors accumulate through the switched stack and the final x_T points in a different direction from the native one. Nevertheless, that displacement—$\sim 21°$ on ConvNeXt-T, $\sim 11°$ on the weight-tied model (Section 3.3.4)—still lands in a zone that stage 4 and the head can classify: without training, refining at fixed T costs only 0.6–0.9 top-1 points on untied models and 0.2 on the weight-tied one (Table 1). The next paragraph asks whether a light finetune of that decoder closes the rest (already yes at $R{=}3$; $R{=}100$ pending).

Finetuning stage 4 and the head after unrolling. As a complementary check, we freeze stem and stages 1–3 and finetune only stage 4 and the classification head on the unrolled transport (horizon $T{=}9$ conserved). On ConvNeXt-T with $R{=}3$ ($\Delta t{=}1/3$) this recovers 80.93%

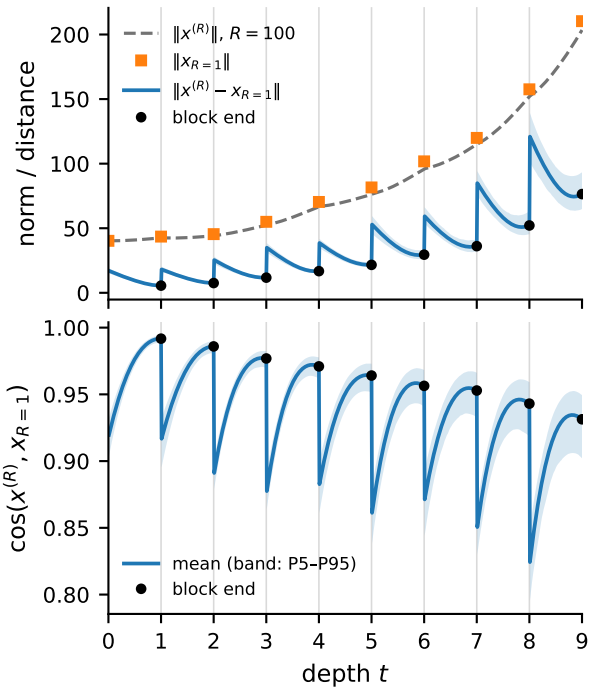


Figure 5. ConvNeXt-T, R=100, Δt=0.01, against the native R=1 path. **Top:** $\|x^{(R)}(t) - x_{R=1}\|$ (solid), with the state norms of both paths for scale (dashed: R=100; squares: R=1). **Bottom:** $\cos(x^{(R)}, x_{R=1})$. Mean and P5–P95 over 500 images; massive channel ignored; dots mark block ends. The norms nearly coincide while the block-end gap grows with depth: the mismatch is mostly in angle.

top-1—within 0.04 points of the untied baseline (80.97%).[1] The same protocol at the fine step of Figure 5 is running now; we report XX% top-1 for ConvNeXt-T at R=100 (Δt=0.01) and XX% for the weight-tied model at R=100 (900 shared steps) after 10 epochs at learning rate 10^{-6} (`finetune_s4_r100_convnext.sh`, `finetune_s4_d900_shared_convnext.sh`). Preliminary runs suggest the gap closes essentially to baseline, consistent with the R=3 result: the transport changes; a light decoder adaptation re-reads it.

Takeaway. Three facts summarise the looping probe. *(i) Discretisation.* Refining the Euler step at fixed T does *not* reproduce the native one-step state: at R=100, after removing the massive channel, the stage-3 endpoint has relative ℓ_2 gap ≈ 0.36 to the parallel R=1 path (Figure 5) and cosine ≈ 0.93 ($\sim 21°$), so the mismatch is mostly angular rather than a pure rescaling of $\|x\|$. The weight-tied stage does not accumulate this error: its cosine to the R=1 path settles at ≈ 0.98 (relative gap ≈ 0.20) instead of falling block after block (Section 3.3.4). *(ii) Continuous limit.* That gap is *not* solver noise: Euler and RK4 at the same fine step, and Euler R=100 vs. R=200, agree on x_T to relative distance $\ll 10^{-2}$ (Appendix C). A continuous limit of the piecewise residual field therefore exists, but it is not the trained R=1 discretisation. Neither fact is specific to training. A random-initialisation control (Appendix D) shows that the blocks of all five architectures, random or trained, converge to a single $\hat{x}_T$ under refinement. How far the one-step path lies from that limit is governed mainly by how far each block moves the state, $\|f_i(x_i)\|/\|x_i\|$: weights trained to be used once produce steps of 30–90% of $\|x\|$ and sit 10–21° from their limit, too far for one Euler step to track the flow. At the same relative step, training moves that distance in an architecture-dependent direction: trained Swin-T and ConvNeXt-T are closer to their limit than a random field (19.5° vs. 47.6°, 21.1° vs. 30.2°), the weight-tied model is level with it (10.9° vs. 11.9°), and the ResNets are farther. What distinguishes the trained networks is therefore not *whether* the refinement converges but *how* the trajectory is built: one shared block applied throughout the stage versus nine different blocks applied once each. Section 3.3 contrasts the two regimes. *(iii) Accuracy.* Top-1 nevertheless barely moves under refinement without training, because the head cares about class margins, not isometry of the full feature space. Finetuning stage 4 and the head recovers 80.93% at R=3 and XX% / XX% for ConvNeXt-T / weight-tied at R=100 —essentially the baseline once the decoder adapts. In short: residual blocks admit a well-defined fine-step flow, as any Lipschitz residual field does; their single unit step is too large to be a consistent solver of that flow; the flow nonetheless retains the network's function (the limit keeps $\approx 80\%$ top-1, which a random field cannot show), accuracy survives because the head is tolerant, and a light S4+head adaptation closes almost all of the remaining gap.

Influenced by continuous-in-depth models (Queiruga et al., 2020; Chang et al., 2018), we also tried replacing the piecewise-constant schedule $\theta_t = \theta_{\lfloor t \rfloor}$ by a bilinear blend of consecutive block weights. It performs relatively badly on every architecture (e.g. -6.6 points on ConvNeXt-T at R=128 vs. plain); full numbers and the residual-norm trajectories are in Appendix F.

3.2.2. Piecewise vs. Smooth Residual Fields

Figure 1 showed the two regimes on individual trajectories; Figure 2 (right) makes the same contrast on the residual-field norm over 500 images. In the non-tied ConvNeXt, $\|f_k(x)\|$ is approximately constant inside each block and jumps at integer depths: a *smooth function by parts*, i.e. an autonomous field on each unit interval $[i, i+1]$ with discontinuous switches of θ between blocks (Equation (3)).

[1] Script: `scripts/finetune_s4_r_convnext.py`. For contrast, R=3 with Δt=1 (broken horizon T=27) collapses to 41% *without* finetuning (`convnext_r3_quick`).

In the weight-tied model the same shared f is applied for the whole stage, so the residual norm evolves as one continuous curve that rises and then decays (Equation (9)). Markers at $R{=}1$ sit on those curves; refining to $R{=}100$ fills in the intra-block path without changing its global shape. This is the geometric counterpart of the accuracy tables: refining a switched stack converges to a slightly worse continuous limit, while refining a shared field stays near the trained accuracy as long as T is held fixed. Sections 3.3 and 3.4 analyse these trajectories in more detail—in particular, how untied blocks redirect the state between integer depths even though each intra-block piece remains integrable.

3.3. Geometrical analysis of feature trajectories

In this section we analyze the geometry of the trajectories regarding the weight-tied convnext vs the standard convnext. We also compare the standard convnext with resnet and swin, whose properties are very similar since they are non weight-tied and the network learned to make the blocks perform an unique functionality of one step, so there is no reason for the network to learn a more complex dynamic.

3.3.1. Massive activations excluded from analysis

After analyzing some trajectories on our first aproach, they seemed too homogeneos in convnext and swin cases, so we analyzed the features and founded the so called massive activations (Sun et al., 2024). In convnext shared and non-shared there is one channel that grows indifinetly. We did a similar analysis and concluded the same as (Sun et al., 2024)

3.3.2. Trajectories of weight-tied convnxt vs standard convnext

3.3.3. Trajectory analysis of weight-tied convnext

Inpired by some of the metrics of (Pandey et al., 2026) we perform an analysis of the trajectories ignoring the massive channels of Convnext and the massive tokens of swin. Resnet does not show signs of massive activations.

In weight-tied convnext we can see in Figure 2 (right) and Figure 1 (bottom left) the norm of the velocity, that is $f(x(t))$ that has a shape of a bell curve. This information only shows if the point is moving or not, but it doesn't tell if it rotates or move towards the same direction. That's why we plot several new metrics:

Acceleration

Here we decompose the acceleration vector into the sum of two components: The tangential and normal accelerations with respect to the velocity. Note, that in a high-dimentional space of 300,000 dimensions (the case of Stage 3 in convnext), there are 299.999 dimentions that are normal with the velocity, and only 1 that is tangent. The tangent component can be negative or positive with respect to the velocity that is, it reduces or increases the current velocity in this 1D line, but we cna only analyze the norm of the normal acceleration. Even in a such space, that tangential acceleration has a big representation.

We can see

Angular acceleration Trajectory curvature

The shape of the trajectory

With all this tools, we conclude that the trajectory at the start rotates at high speeds, giving the trajectory a high curvature, but with low velocity norm, that implies that it does small steps changing constantly of direction. The model is computing the direction where it has to accelerate the point to.

After that, the model starts to accelerate towards one direction, angular speed decreases, norm of angular acceleration increases due to the reduction of velocity the result is a major reduction on the curvature of the trajectory, while tangential acceleration hits its maximum. The model is expanding the most important features.

Finally, angular acceleration decreases, tangential acceleration rapidly decreases, becoming negative, the velocity decreases, the angular velocity increases again, and curvature raises again. We have the same scenario as in the first phase. The model is refining features again.

We observe that for the 500 images we analyzed, the trajectory is mostly of this form. But in the variance analysis, the velocity has its maximum variance on the second phase since the model is pushing the features of different classes apart from the origin in different directions.

3.3.4. Refinement error: accumulated vs. bounded

Figure 6 repeats the comparison of Figure 5 for the weight-tied network. Both models start the same way: after the first native step the cosine to the $R{=}1$ path is 0.99. They then diverge. In the standard ConvNeXt-T the block-end cosine keeps falling, to 0.93 at $t{=}9$: each block receives an input that has already drifted and adds its own discretisation error on top. In the weight-tied model it settles at ≈ 0.98 after two or three steps and stays there for the rest of the stage, so a single autonomous field keeps the refinement error bounded instead of accumulating it. The norms also behave differently: in ConvNeXt-T they nearly coincide (203 vs. 210), whereas the weight-tied refined path ends about 7% shorter (427 vs. 458), so part of its smaller gap is radial.

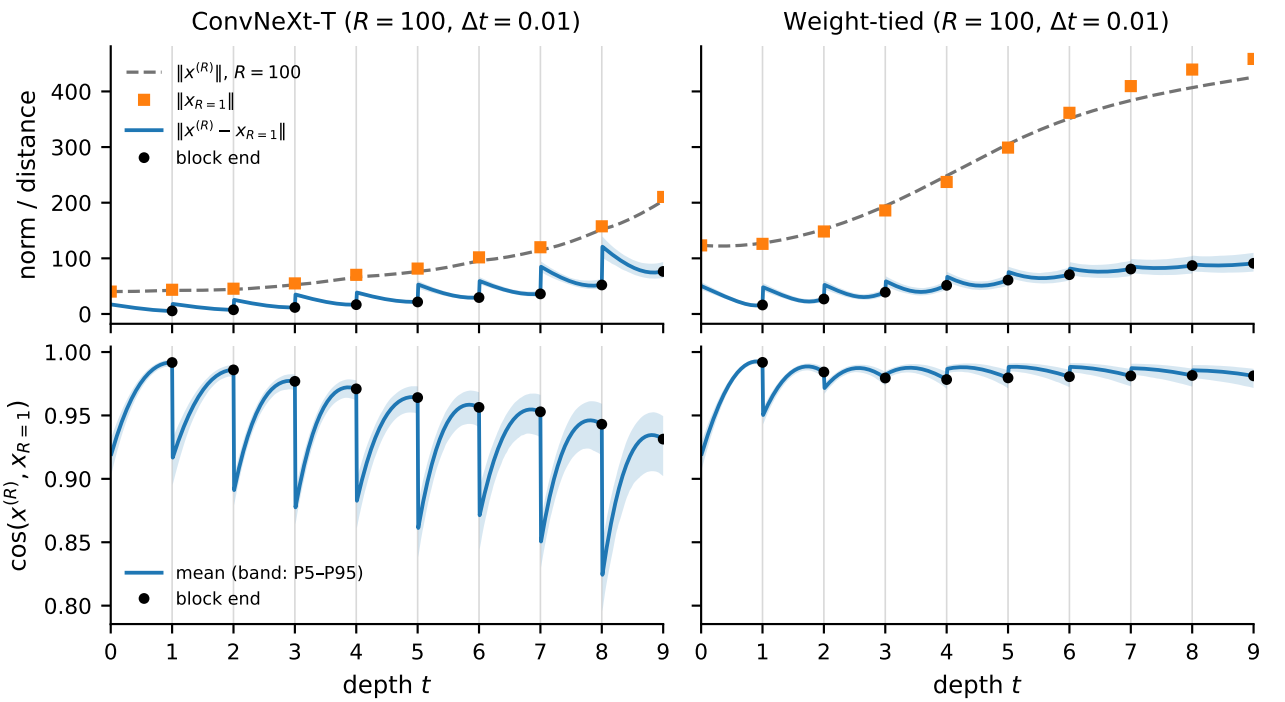


Figure 6. Refined vs. native (R=1) stage-3 path for the standard (left) and weight-tied (right) ConvNeXt-T. **Top:** $\|x^{(R)} - x_{R=1}\|$ (solid) with both state norms for scale (dashed: refined; squares: native). **Bottom:** $\cos(x^{(R)}, x_{R=1})$. Mean and P5–P95 over the same 500 images; massive channel ignored; dots mark the end of each native step. The switched stack accumulates angle at every block; the shared field saturates.

3.3.5. Integrating beyond the trained horizon

There is big difference between weight-tied convnext and standard convnext: First one reaches a velocity plateau where the input just grows indifinetly in the same direction.

In standard convnext, swin and resnet are different, we used R=100 with euler step 1, since the blocks use different parameters and the network was not trained to reuse params, it has a similar dynamics as the R=100 with Euler step of 0.01 but with more magnitude.

3.4. Are over-parametrized networks undoing what the previous layers have done?

Section 3.1 showed that one shared stage-3 block nearly matches nine independent ones (-0.61 top-1 points, 33% fewer parameters). A natural explanation is that the untied stack is over-parametrised along depth: consecutive blocks need not push the state in a coherent direction, and part of what block $k{+}1$ does can cancel what block k just did. We test that claim directly on feature trajectories, using the same 500 ImageNet validation images (one per class) as in Section 3.3, stage 3 only, with the massive channel removed (Section 3.3.1).

Setup. For each image we integrate the stage from t=0 to t=9 at native resolution (R=1) and under Euler refinement (R=100, Δt=0.01), and reduce every spatial feature coordinate whose net displacement $|\Delta_i| = |x_i(9) - x_i(0)|$ is above the per-image median. We partition the trajectory into the nine unit intervals $[k, k{+}1]$ (native blocks in ConvNeXt-T; the same time bins in the weight-tied model) and write $\mathrm{d}x_k$ for the displacement of a coordinate over interval k. The scalars we report are means $\pm$ std over the 500 images (Table 2):

- *straightness*: $\|\sum_k \mathrm{d}x_k\| \,/\, \sum_k \|\mathrm{d}x_k\|$ in feature space (path length vs. net displacement);
- *wasted motion*: $1 - \sum_i |\mathrm{net}_i| / \sum_{k,i} |\mathrm{d}x_{k,i}|$, the fraction of per-coordinate travel that cancels before t=9;
- $\cos(\mathrm{d}x_k, \mathrm{d}x_{k+1})$: mean cosine between consecutive block displacements;
- *undo fraction*: when $\mathrm{sign}(\mathrm{d}x_k)$ flips at the next block, the cancelled mass $\min(|\mathrm{d}x_k|, |\mathrm{d}x_{k+1}|)$ normalised by $\sum |\mathrm{d}x_k|$;
- *zig-zag share*: mass-weighted share of coordinates that reverse at least twice across the nine blocks;

- *sign flips at* $t \in \mathbb{N}$: fraction of consecutive steps whose sign changes exactly at integer depths (block boundaries).

Results. Table 2 and Figure 7 give a consistent picture. Under refinement (R=100), the weight-tied model is nearly monotone along depth: straightness 0.78, wasted motion 0.19, consecutive-block cosine 0.92, undo fraction 0.02, and zig-zag share 0.14. Sign changes at integer times are essentially absent (0.00). ConvNeXt-T, with nine independent blocks, sits at the opposite extreme: straightness 0.50, wasted motion 0.48 (almost half of the travel cancels), cosine 0.17, undo fraction 0.25 ($\approx 12\times$ the shared value), and zig-zag share 0.90. Nearly half of the sign flips (0.45) fall exactly on block boundaries—the discontinuities of the switched field in Section 3.2.2, not an artefact of a coarse step. Native resolution (R=1) barely changes the ConvNeXt-T numbers; refining only makes the shared field look cleaner (undo $0.05 \rightarrow 0.02$), which matches the bounded refinement error of Section 3.3.4.

Functional progress along depth. Geometry shows wasted motion; a complementary probe asks when the stage 3 state becomes usable by the rest of the network. For each k=1, . . . , 9 we stop after the k-th residual block and classify with the model's own trained downsample, stage 4 and head—no remaining stage 3 blocks (Figure 8; full ImageNet validation). Under weight tying the top-1 rises early and smoothly: 33.6% after one block, 72.2% after three, and 78.9% by block 5, already close to the final 80.3%. On standard ConvNeXt-T the same frozen decoder stays weak for longer (11.5%, 33.2%, 66.9% at those depths) and only approaches the final 81.0% in the last few blocks. Mid-stage accuracy is not a claim that early features lack information—the head was trained on x_T—but it does show that the tied path enters the decoder's operating region steadily, whereas the untied path aligns with that region late, consistent with a transport that rewrites features before committing.

Reading. Untied stage 3 does not implement nine steps of one transport: consecutive blocks often point in weakly related or opposing directions and cancel a large fraction of each other's work. The weight-tied network cannot cancel itself in that way—a single autonomous field has no "next block" with different weights—and the geometry shows almost no undoing. Figure 8 makes the same contrast in function space: tied depth builds decoder-ready features gradually, while untied depth postpones most of the usable progress. That is why sharing the nine blocks costs less than one top-1 point (Section 3.1): much of the untied capacity along depth is spent rewriting features rather than accumulating a coherent displacement toward x_T. The residual accuracy gap to the baseline is then more plausibly attributed to the small, non-cancellable specialization that independent θ_k still provide, not to a fundamentally different dynamical mechanism.

3.5. Shared field with per-block deltas

4. Discussion

The experiments of Section 3 are one argument, not five independent facts. Table 3 collects the numbers that carry it.

One shared residual map, applied nine times, already carries almost all of the accuracy of an untied ConvNeXt-T stage 3 (-0.61 top-1; the sweep checkpoint in Table 3 is 80.34 vs. 80.95). Sharing every stage still reaches 79.01 with 70% fewer parameters. That is possible because the nine independent blocks do not implement nine steps of one transport. Almost half of their travel cancels before t=9 (undo 0.25, wasted motion 0.48), consecutive displacements are nearly orthogonal (cosine 0.17), and nearly half of the sign flips sit on block boundaries—the switches of $\theta(t) = \theta_{\lfloor t \rfloor}$. A single autonomous field cannot undo itself in that way (undo 0.02, cosine 0.92, straightness 0.78). The leftover six tenths of a point are then the part that independent θ_k still specialise, not evidence of a different dynamical mechanism (Section 3.4). Closing that gap with small per-block corrections on top of the shared field is the content of Section 3.5.

The same contrast appears when the discretisation is refined at inference, with no training. Every architecture we tested—tied or not—converges to a single continuous limit once T is held fixed, and that limit keeps the network's function (gaps 0.2–0.9 points). A random residual field has a continuous limit too (Appendix D); what training adds is that the limit remains a classifier. The native unit-step path is *not* that limit. On ConvNeXt-T the refined endpoint sits $\approx 21^\circ$ from the R=1 state and the mismatch accumulates at every block; on the weight-tied model it saturates at $\approx 11^\circ$ (Section 3.3.4). Breaking the horizon (R repeats at Δt=1) collapses every network to chance. The invariant the head expects is the integration time T=n, not the number of residual applications. In that sense a trained stage is a sum of one-step Euler solves, each accurate enough for the decoder at Δt=1 and each replaceable by a finer integral of the *same* block, but not a Neural ODE whose parameters vary smoothly with depth: bilinear interpolation of consecutive θ_k costs 6–23 points (Appendix F).

Two regimes therefore sit under the same residual algebra $x + f(x)$. Reuse one block throughout the stage and the field is autonomous: speed rises and falls as one curve, refinement error stays bounded, features advance mostly monotonically. Use a new block at every integer and the field is switched: piecewise autonomous inside each unit interval, discon-

Table 2. Pooled stage-3 trajectory statistics over 500 ImageNet images (massive channel ignored; coordinates with $|\Delta_i|$ above the per-image median). Weight-tied vs. standard ConvNeXt-T at native (R=1) and refined (R=100) resolution. Mean ± std across images.

Model	R	straightness	wasted motion	monotone share	$\cos(\Delta x_k, \Delta x_{k+1})$	reversal rate	undo fraction	zig-zag share	sign flips at $t \in \mathbb{N}$	median t_{50}
Weight-tied ConvNeXt	100	0.78±0.02	0.19±0.01	0.73±0.02	0.92±0.01	0.11±0.01	0.02±0.00	0.14±0.02	0.00±0.00	4.09±0.16
ConvNeXt-T	100	0.50±0.01	0.48±0.01	0.21±0.01	0.17±0.01	0.45±0.00	0.25±0.00	0.90±0.00	0.45±0.00	7.21±0.15
Weight-tied ConvNeXt	1	0.72±0.02	0.25±0.02	0.63±0.03	0.77±0.03	0.18±0.01	0.05±0.01	0.34±0.03	0.18±0.01	5.00±0.09
ConvNeXt-T	1	0.50±0.01	0.48±0.01	0.22±0.01	0.16±0.01	0.45±0.00	0.25±0.00	0.90±0.00	0.45±0.00	7.82±0.38

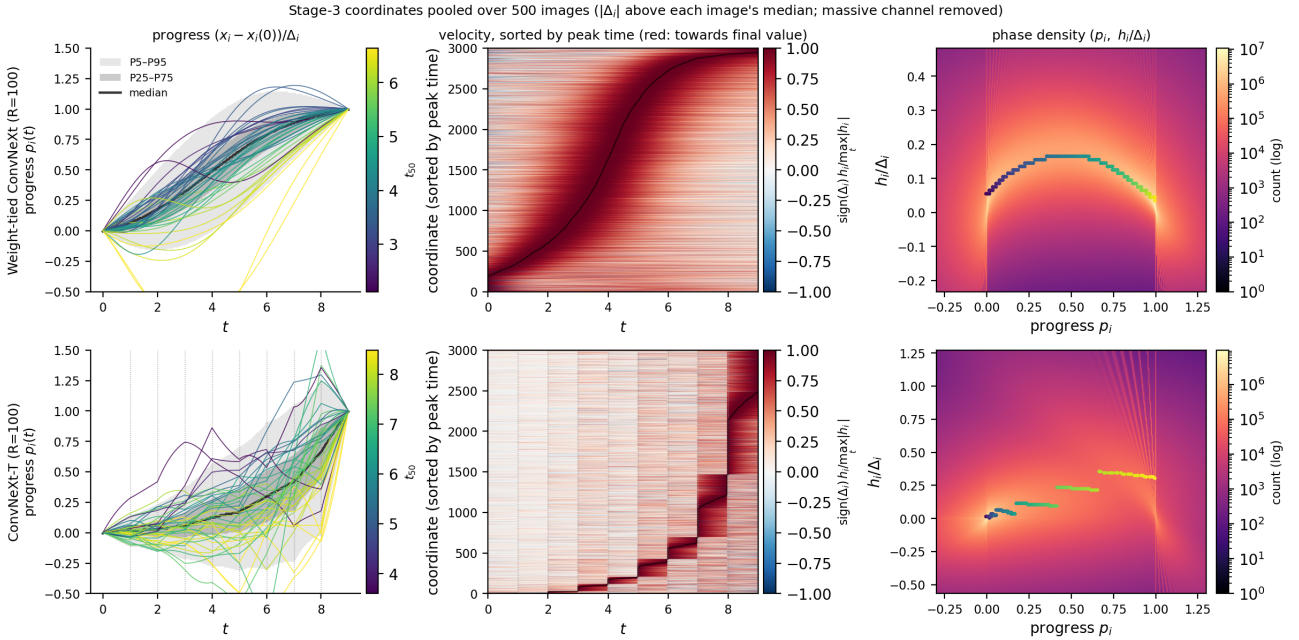


Figure 7. Stage-3 coordinates pooled over 500 images (R=100; massive channel removed; $|\Delta_i|$ above the per-image median). **Top:** weight-tied ConvNeXt; **bottom:** ConvNeXt-T. **Left:** progress $p_i(t) = (x_i(t) - x_i(0))/\Delta_i$ (median and P5–P95; thin lines: random coordinates). **Middle:** velocity toward the final value, coordinates sorted by peak time (red: toward the end state; blue: away). **Right:** phase density $(p_i,\ h_i/\Delta_i)$. The shared field advances mostly monotonically; the switched stack reverses and rewrites features between blocks.

tinuous at the integers, with a sawtooth residual norm and systematic undoing between neighbours (Figures 1, 2 and 7). That is the reading we take into the geometry of Section 3.3. A light adaptation of stage 4 and the head already restores the baseline at R=3 (80.93%), which is consistent with the transport changing while remaining readable; whether the same holds at the fine step of Figure 5 is the remaining decoder check. Whether one can retain the expressivity of independent θ_k while suppressing cancellation—and thus get more accuracy per parameter—is left for future work.

5. Conclusion

References

Ainsworth, S. K., Hayase, J., and Srinivasa, S. Git re-basin: Merging models modulo permutation symmetries. In *International Conference on Learning Representations (ICLR)*, 2023.

Chang, B., Meng, L., Haber, E., Tung, F., and Begert, D. Multi-level residual networks from dynamical systems view. In *International Conference on Learning Representations (ICLR)*, 2018.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 248–255, 2009.

E, W. A proposal on machine learning via dynamical systems. *Communications in Mathematics and Statistics*, 5 (1):1–11, 2017.

Entezari, R., Sedghi, H., Saukh, O., and Neyshabur, B. The role of permutation invariance in linear mode connectiv-

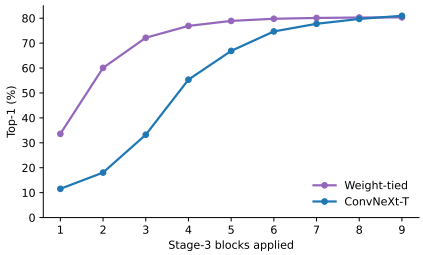


Figure 8. ImageNet top-1 of each model's trained stage 4+head on stage 3 prefixes: after k residual blocks, skip the rest of stage 3 and decode. Weight-tied features become usable earlier and more evenly; ConvNeXt-T concentrates most of the gain toward the end of the stage.

Table 3. Headline measurements on ImageNet-1k. Top-1 and ODE limit from the conserved-horizon sweep of Table 1 (ODE: RK4 at the finest step). Straightness and undo: stage-3 trajectories at R=100 (Table 2). Params and the all-stages run are from Section 3.1. Swin-T and the ResNets are the standard pretrained checkpoints; we did not retrain them.

Model	Params	R=1	ODE	Gap	Str.	Undo
ConvNeXt-T	28.59M	80.95	80.23	−0.72	0.50	0.25
Weight-tied S3	18.97M	80.34	80.12	−0.22	0.78	0.02
All-stages tied	8.68M	79.01	—	—	—	—
Swin-T	—	81.08	80.32	−0.76	—	—
ResNet-50	—	80.34	79.43	−0.91	—	—
ResNet-101	—	81.67	81.05	−0.62	—	—

ity of neural networks. In *International Conference on Learning Representations (ICLR)*, 2022.

Glorot, X. and Bengio, Y. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 249–256, 2010.

Haber, E. and Ruthotto, L. Stable architectures for deep neural networks. *Inverse Problems*, 34(1):014004, 2017.

Hairer, E., Nørsett, S. P., and Wanner, G. *Solving Ordinary Differential Equations I: Nonstiff Problems*, volume 8 of *Springer Series in Computational Mathematics*. Springer, 2 edition, 1993.

He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., and Xie, S. A ConvNet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

Lu, Y., Zhong, A., Li, Q., and Dong, B. Beyond finite layer neural networks: Bridging deep architectures and numerical differential equations. In *International Conference on Machine Learning (ICML)*, 2018.

Pandey, V., Singh, G., and Mahdid, Y. Trajectory geometry of transformer representations across layers, 2026. URL https://arxiv.org/abs/2606.09287.

Queiruga, A. F., Erichson, N. B., Taylor, D., and Mahoney, M. W. Continuous-in-depth neural networks, 2020. URL https://arxiv.org/abs/2008.02389.

Sander, M. E., Ablin, P., and Peyré, G. Do residual neural networks discretize neural ordinary differential equations? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

Sun, M., Chen, X., Kolter, J. Z., and Liu, Z. Massive activations in large language models, 2024. URL https://arxiv.org/abs/2402.17762.

Sun, Q., Pickett, M., Nain, A. K., and Jones, L. Transformer layers as painters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 25219–25227, 2025.

A. Training recipe

We train on ImageNet-1k (Deng et al., 2009) at 224×224. Table 4 lists the full hyper-parameter table, next to the official ConvNeXt-T recipe (Liu et al., 2022). The four models we train from scratch share the same augmentation, regularisation and optimiser; they differ only in peak learning rate, warmup length, minimum learning rate and stochastic depth, as marked in the table. The official ConvNeXt-T uses batch size 4096 and base learning rate 4×10^{-3}. Linear scaling to our batch of 1024 gives 1×10^{-3}, which is the peak we use for the weight-tied and all-stages runs. The untied ConvNeXt-T (and its drop-path-free sibling) use half of that, 5×10^{-4}, with the official 20-epoch warmup.

Weight decay is applied only to convolution and linear weights (tensors of rank ≥ 2). LayerNorm gain and bias, convolution/linear bias and LayerScale γ are decay-free, following the ConvNeXt / DeiT convention. Convolution and linear weights are initialised with truncated normal $\mathcal{N}(0, 0.02^2)$; LayerScale uses $\gamma{=}10^{-6}$. There is no layer-wise learning-rate decay, no gradient clipping, no mixed precision and no exponential moving average of the weights. The learning-rate schedule is a linear warmup (starting at 10^{-6}) followed by cosine decay to the listed minimum, advanced once per optimiser step.

Mixup and CutMix are applied to every training batch (switch probability 0.5), and the loss is soft-target cross-entropy on the mixed labels. Per-image augmentation is the timm ConvNeXt stack: random resized crop, horizontal flip, RandAugment `rand-m9-mstd0.5`, color jitter 0.4 and random erasing (probability 0.25, pixel mode). Validation uses the standard centre crop. Stochastic depth, when enabled, is ramped linearly from 0 to the listed peak across the 18 residual blocks.

Stage-4 + head finetune after unrolling. The decoder-adaptation probe of Section 3.2 freezes stem and stages 1–3, unrolls stage 3 at inference resolution ($R{=}100$, $\Delta t{=}0.01$, $T{=}9$; 900 residual steps on the weight-tied model) and trains only stage 4, the global pool and the classification head. We run 10 epochs at a constant learning rate 5×10^{-6} (no warmup, no cosine decay), with the same optimiser, weight decay, batch size and augmentation as Table 4.

B. Full Midpoint and RK4 refinement tables

We use the same R (repetitions) as in the main text: $\Delta t{=}1/R$ so that $T{=}n$ is conserved ($n{=}9$ for ConvNeXt stage 3). Tables 5–8 report plain interpolation for Euler (RK1), Midpoint (RK2) and RK4 on each pretrained model. Table 9 gives the same solvers for the weight-tied ConvNeXt-T at fixed horizon $T{=}9$ ($9R$ applications of the shared block). In every case the three integrators converge to the same continuous limit once the step is fine enough; at $R{=}1$, Mid and RK4 already sit on that limit.

C. Solver agreement at the stage-3 endpoint

Section 3.2 asks whether a fine Euler step recovers the native $R{=}1$ state. A related question is whether two fine discretisations of the *same* residual field agree with each other—i.e. whether a continuous limit exists at all. Table 10 compares stage-3 endpoints x_T on the same $N{=}32$ ImageNet validation images (one per random class, seed 0), after zeroing the massive channel used in the main-text metrics. Relative distance is $\|x_a - x_b\|/\|x_b\|$; cosine is $\cos(x_a, x_b)$.

Euler and RK4 (and Euler $R{=}100$ vs. $R{=}200$) collapse onto the same x_T, while both remain far from the trained one-step path. RK4 reaches that limit already with the nine unit steps of the native schedule; the native Euler path does not. Midpoint at $R{=}1$ matches the limit in top-1 (80.23%, Table 5) while its state is still 12% away, a reminder that accuracy is a coarse probe of x_T.

D. Random-initialisation control for the continuous limit

Appendix C shows that the trained stage 3 has a continuous limit. That alone is weak evidence. Every residual field we study is a composition of Lipschitz maps (convolutions, linear layers, normalisation, GELU/ReLU, softmax attention), so the ODE $\dot{x} = f_i(x)$ has a unique solution on $[0, T]$ and Euler converges to it at first order, whatever the weights (Hairer et al., 1993). We check this empirically on all five architectures and use random fields of controlled strength as a reference scale for the trained ones.

Setup. Every convolution and linear layer is drawn with Xavier-normal initialisation (Glorot & Bengio, 2010) (gain 1, zero bias, seed 0); normalisation layers keep their default parameters, and ConvNeXt LayerScale is set to $\gamma{=}1$ (its default 10^{-6} makes $f \approx 0$). The stage-3 blocks are then re-drawn with Xavier gain $g \in \{0.25, 0.5, 1, 2, 4, 8\}$, so the stage-3 input is identical across gains and only the field strength changes. For ResNet the field is $f_i(x) = \mathrm{block}_i(x) - x$, which includes the ReLU applied after the addition; for Swin one block contains both residual sub-layers (attention and MLP), as in the main text. For each network we integrate stage 3 to $T{=}n$ and compare the endpoint x_T of Euler at $R \in \{1, 10, 100, 1000\}$ and of RK4 at $R{=}1$ against RK4 at $R{=}100$, on the $N{=}32$ images of Appendix C. The field strength is summarised by the mean relative step of the native path,

$$\rho \;=\; \frac{1}{n}\sum_{i=0}^{n-1}\frac{\|f_i(x_i)\|}{\|x_i\|}, \qquad x_{i+1} = x_i + f_i(x_i), \quad (10)$$

Table 4. ImageNet-1k training settings. Official ConvNeXt-T from Liu et al. (2022); the remaining columns are the four models we train. CNX-T: untied baseline. Tied: stage 3 shares one block (D=9). All: one shared block per stage. dp0: untied ConvNeXt-T with stochastic depth off.

	Official	CNX-T	Tied	All	dp0
training epochs	300	300	300	300	300
warmup epochs	20	20	10	10	20
optimizer	AdamW	AdamW	AdamW	AdamW	AdamW
base learning rate	4×10^{-3}	5×10^{-4}	1×10^{-3}	1×10^{-3}	5×10^{-4}
min. learning rate	—	10^{-6}	10^{-7}	10^{-7}	10^{-6}
weight decay	0.05	0.05	0.05	0.05	0.05
optimizer momentum	β=(0.9, 0.999)	β=(0.9, 0.999)	β=(0.9, 0.999)	β=(0.9, 0.999)	β=(0.9, 0.999)
batch size	4096	1024	1024	1024	1024
learning rate schedule	cosine	cosine	cosine	cosine	cosine
warmup schedule	linear	linear	linear	linear	linear
layer-wise lr decay	None	None	None	None	None
RandAugment	(9, 0.5)	(9, 0.5)	(9, 0.5)	(9, 0.5)	(9, 0.5)
Mixup α	0.8	0.8	0.8	0.8	0.8
CutMix α	1.0	1.0	1.0	1.0	1.0
random erasing	0.25	0.25	0.25	0.25	0.25
color jitter	0.4	0.4	0.4	0.4	0.4
label smoothing	0.1	0.1	0.1	0.1	0.1
stochastic depth	0.1	0.1	0.1	0.1	0
LayerScale init	10^{-6}	10^{-6}	10^{-6}	10^{-6}	10^{-6}
gradient clip	None	None	None	None	None
exp. moving average	None	None	None	None	None
mixed precision	—	off	off	off	off

Table 5. ConvNeXt-T, plain, top-1 (%).

R	Euler	Mid	RK4
1	80.95	80.23	80.23
2	80.79	80.23	80.23
4	80.53	80.23	80.23
10	80.36	80.23	80.23
32	80.30	80.23	80.23
128	80.23	80.22	80.23

Table 6. Swin-T, plain, top-1 (%).

R	Euler	Mid	RK4
1	81.08	80.29	80.33
2	80.90	80.34	80.31
4	80.65	80.33	80.32
10	80.47	80.32	80.31
32	80.38	80.32	80.32
128	80.34	80.31	80.32

i.e. how far a single block moves the state relative to its size. Because ρ grows with g at a different rate in each architecture (about g^3 for ResNet, whose bottleneck has three unnormalised convolutions), we compare trained and random networks at matched ρ: for each architecture we add one gain $g^\star$ chosen so that ρ matches the trained network. Trained ConvNeXt rows zero the massive channel (Section 3.3.1); ResNet and Swin have no channel of comparable dominance at stage 3 (zeroing the largest one changes every entry by less than $0.6°$), so all rows except those use all channels.

Results. Table 11 supports two observations.

(i) The limit exists for every architecture. For every row, trained or random, the Euler error falls by roughly $10\times$ per decade of R, the first-order rate (trained Swin-T is slower between R=10 and R=100), and Euler at R=1000 is within $0.1°$ of the reference. The same holds at the larger gains not shown, with two numerical exceptions that confirm the mechanism rather than contradict it. ResNet at $g \geq 4$ overflows in float32: without a normalisation inside the block, $\|f_i(x)\|$ grows with $\|x\|$, so the state grows exponentially ($\rho \approx 18$ per block at g=4). Swin at g=8 is still $15.7°$ away at R=1000: the attention logits are so large that the softmax is nearly a hard selection, the field becomes very stiff, and a finer step is needed. Convergence of the refinement to a single $\hat{x}_T$ is therefore a property of the residual parameterisation, not something learned.

(ii) The native gap is of the order set by ρ, and training moves it in an architecture-dependent direction. Within each architecture the distance from the single-step path (R=1) to the limit grows monotonically with ρ, and with $\rho \approx 0.3$–0.9 every trained network lies 10–21° away: one Euler step of size 1 is far from tracking the flow. At matched ρ, how-

Table 7. ResNet-50, plain, top-1 (%).

R	Euler	Mid	RK4
1	80.34	78.94	79.42
2	80.06	79.41	79.43
4	79.84	79.44	79.43
10	79.61	79.44	79.43
32	79.50	79.43	79.44
128	79.45	79.43	79.43

Table 8. ResNet-101, plain, top-1 (%).

R	Euler	Mid	RK4
1	81.67	80.95	81.06
2	81.58	81.07	81.04
4	81.38	81.06	81.05
10	81.22	81.05	81.05
32	81.12	81.05	81.04
128	81.06	81.04	81.05

ever, the trained and random networks differ, and not in a consistent direction. Trained Swin-T and ConvNeXt-T are *closer* to their limit than a random field of the same strength (19.5° vs. 47.6° and 21.1° vs. 30.2°), the weight-tied model is level with it (10.9° vs. 11.9°), and both ResNets are *farther* (17.0° vs. 12.0° and 15.3° vs. 8.9°). RK4 at $\Delta t{=}1$ follows the same split. Training thus changes how well a single step approximates the flow, but there is no universal tendency for single-use blocks to be better (or worse) one-step solvers than random ones. Script: `scripts/random_init_convergence.py`.

E. Block permutation, duplication and substitution

F. Bilinear weight interpolation

Continuous-in-depth work often assumes a smooth path in weight space between consecutive blocks (Queiruga et al., 2020; Chang et al., 2018),

$$\theta(t) \;=\; (1-\alpha)\,\theta_i + \alpha\,\theta_{i+1}, \qquad \alpha = t - i. \tag{11}$$

Replacing the plain schedule $\theta_t = \theta_{\lfloor t \rfloor}$ by (11) hurts every model we tested: at $R{=}128$ the drop vs. plain is between 6.6 and 23.3 points (Table 12). Consecutive blocks are not neuron-aligned, so the midpoint of their weights is not a functional intermediate (Entezari et al., 2022; Ainsworth et al., 2023).

Figure 9 shows the ConvNeXt residual-field norm under plain vs. bilinear schedules at $R{=}100$. Plain keeps the piecewise-constant staircase of Figure 2 (right); bilinear adds high-frequency oscillation between block boundaries

Table 9. Shared ConvNeXt-T, fixed $T{=}9$ ($\Delta t{=}1/R$; $9R$ shared steps), top-1 (%).

R	Steps	Δt	Euler	Mid	RK4
$1/9$	1	9	47.06	0.61	2.70
$2/9$	2	4.5	5.85	10.42	47.98
$3/9$	3	3	37.45	54.47	71.37
$4/9$	4	2.25	67.83	73.57	78.90
$5/9$	5	1.8	77.23	78.70	79.97
$6/9$	6	1.5	79.48	79.85	80.10
$7/9$	7	1.29	80.19	80.12	80.11
$8/9$	8	1.13	80.38	80.21	80.11
1	9	1	80.34	80.19	80.11
$4/3$	12	0.75	80.35	80.14	80.12
2	18	0.5	80.27	80.12	80.12
$32/9$	32	0.28	80.22	80.11	80.12
$128/9$	128	0.07	80.14	80.12	80.12
$1024/9$	1024	0.009	80.12	80.12	80.12
$10000/9$	10000	0.0009	80.12	—	—

Table 10. Pairwise stage-3 endpoint agreement (mean over $N{=}32$; ignore top-1 channel). Horizon $T{=}9$ in every row.

Pair (a vs. b)	rel. ℓ_2	cos	angle
ConvNeXt-T			
Euler $R{=}100$ vs. RK4 $R{=}100$	0.004	1.000	0.25°
Euler $R{=}100$ vs. Euler $R{=}200$	0.002	1.000	0.12°
Euler $R{=}100$ vs. Euler $R{=}1$	0.36	0.93	21°
Mid $R{=}1$ vs. Euler $R{=}100$	0.12	0.993	6.6°
RK4 $R{=}1$ vs. Euler $R{=}100$	0.010	0.9999	0.5°
Weight-tied ConvNeXt-T ($9R$ shared steps)			
Euler $R{=}100$ vs. RK4 $R{=}100$	0.002	1.000	0.1°
Euler $R{=}100$ vs. Euler $R{=}200$	0.001	1.000	0.05°
Euler $R{=}100$ vs. Euler $R{=}1$	0.20	0.98	11°
RK4 $R{=}1$ vs. RK4 $R{=}100$	0.003	1.000	0.16°

while preserving the same global growth of $\|f\|$. Bands are P5–P95 across 500 validation images (same suite as Figure 2).

Table 11. Angle between x_T and the RK4 R=100 reference (T=n, mean over N=32). g: Xavier gain of the stage-3 blocks; ρ: mean relative step (10); *: gain matched to the trained ρ. Rows sorted by ρ.

Init	ρ	Euler			RK4
		R=1	R=10	R=100	R=1
ConvNeXt-T (nine blocks)					
g=0.5	0.13	1.5°	0.15°	0.02°	0.00°
g=1	0.41	15.8°	1.6°	0.16°	0.07°
Trained	**0.54**	**21.1°**	**2.4°**	**0.25°**	**0.53°**
$g^\star$=1.35	0.56	30.2°	3.4°	0.34°	0.92°
g=2	0.77	47.0°	6.7°	0.69°	8.4°
Weight-tied ConvNeXt-T (one block)					
g=0.5	0.12	2.7°	0.27°	0.03°	0.00°
$g^\star$=1	0.30	11.9°	1.3°	0.13°	0.05°
Trained	**0.31**	**10.9°**	**1.0°**	**0.10°**	**0.16°**
g=2	0.63	27.1°	4.3°	0.46°	5.9°
ResNet-50 (n=5)					
g=0.5	0.04	0.4°	0.03°	0.00°	0.01°
g=1	0.34	7.0°	0.66°	0.07°	0.24°
$g^\star$=1.15	0.51	12.0°	1.2°	0.12°	0.48°
Trained	**0.52**	**17.0°**	**1.4°**	**0.15°**	**0.65°**
g=2	2.29	40.6°	7.8°	0.83°	6.6°
ResNet-101 (n=22)					
g=0.5	0.05	0.7°	0.05°	0.01°	0.02°
Trained	**0.30**	**15.3°**	**1.4°**	**0.15°**	**0.74°**
$g^\star$=0.95	0.30	8.9°	0.83°	0.08°	0.29°
g=1	0.35	10.3°	0.97°	0.10°	0.36°
g=2	2.33	39.6°	5.9°	0.63°	6.4°
Swin-T (n=6)					
g=0.5	0.15	1.7°	0.17°	0.02°	0.00°
g=1	0.48	17.2°	1.8°	0.18°	0.10°
Trained	**0.87**	**19.5°**	**4.3°**	**1.02°**	**2.7°**
$g^\star$=1.8	0.89	47.6°	7.1°	0.74°	6.6°
g=2	0.98	52.7°	8.7°	0.91°	10.5°

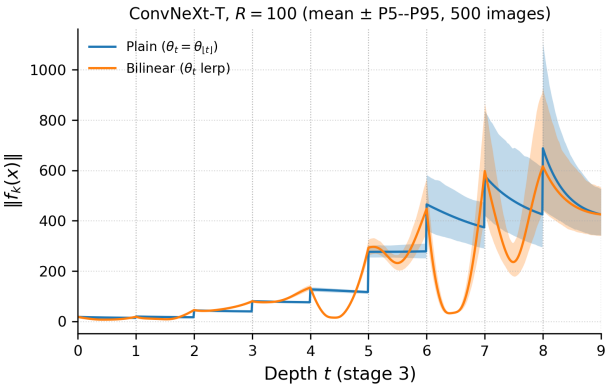


Figure 9. ConvNeXt-T residual norm along stage 3 at R=100: plain vs. bilinear weight interpolation (mean and P5–P95 over 500 images).

Table 12. Bilinear weight interpolation, Euler, top-1 (%).

R	CNX-T	CNX-T†	Swin-T	R-50	R-101
1	80.96	79.27	81.08	80.34	81.67
2	78.94	72.43	78.93	74.95	72.55
4	76.16	63.13	75.42	70.60	62.42
10	74.42	58.75	73.28	68.58	58.73
128	73.66	57.44	72.14	67.77	57.73
Δ vs plain @128	−6.6	−19.6	−8.2	−11.7	−23.3

†ConvNeXt-T trained without drop path.